MODULE 15

Testing & Optimization

JUnit, Espresso

Phase 03 | Android Development with Kotlin

Testing Strategy and Local Unit Tests

- Apply the test pyramid to decide which kind of test each piece of the PNC Mobile app needs
- Choose between fakes, stubs, and mocks and justify the choice
- Write deterministic JUnit tests using Arrange, Act, Assert with behavior-based names
- Cover edge cases with parameterized tests and test data builders



©Logical Imagination Group LLC

The Test Pyramid for Mobile

- Local unit tests run on the JVM in milliseconds: the wide base
- Integration tests combine real components such as Room plus a repository
- UI and end-to-end tests run on a device or emulator: few, slow, valuable
- Source sets: src/test runs locally, src/androidTest runs on a device

Layer	Test Type	Source set	Speed
Mappers, Validators	Local unit	src/test	ms
UseCases, ViewModels	Local unit + fakes	src/text	ms
Repository + Room	Integration	src/androidTest	100s of ms
Retrofit + interceptors	Integration	src/test (MockWebServer)	ms
Screens (Compose)	UI Test	src/androidTest	seconds
Login to transfer journey	End to end	src/androidTest	10s of seconds



©Logical Imagination Group LLC

Anatomy of a Good Unit Test

Arrange, Act, Assert in three visible blocks

Name tests after behavior: returnsErrorWhenAmountExceedsBalance

One behavior per test; a failure should point to one cause

Deterministic: no real clock, network, randomness, or shared state



©Logical Imagination Group LLC

Anatomy of a Good Unit Test (code)

```
class MoneyTest {
    @Test
    fun addingTwoAmountsSumsCentsExactly() {
        val tenCents = Money(10)
        val twentyCents = Money(20)
        val total = tenCents + twentyCents
        assertEquals(Money(30), total)
    }

    @Test
    fun formatsUsingGivenLocale() {
        assertEquals("$1,234.50", Money(123450).format(Locale.US))
    }
}
```



©Logical Imagination Group LLC

Fakes, Stubs, and Mocks

Stub returns canned answers; a fake is a lightweight working implementation

Mock records calls so you can verify interactions

Prefer fakes for repositories: tests survive refactoring

Use MockK to verify a boundary call that has no observable result



©Logical Imagination Group LLC

Fakes, Stubs, and Mocks (code)

```
class FakeAccountsRepository : AccountsRepository {
    private val accounts = MutableStateFlow<List<Account>>(emptyList())
    var refreshResult: ApiResult<Unit> = ApiResult.Success(Unit)
    var refreshCount = 0
    fun emit(list: List<Account>) { accounts.value = list }
    override fun observeAccounts(): Flow<List<Account>> = accounts
    override suspend fun refresh(): ApiResult<Unit> { refreshCount++; return refreshResult }
}

@Test fun sendsAnalyticsOnce() {
    val analytics = mockk<Analytics>(relaxed = true)
    TransferScreenLogic(analytics).onSubmit()
    verify(exactly = 1) { analytics.track("transfer_submitted") }
}
```



©Logical Imagination Group LLC

Parameterized Tests

- Boundary values find most bugs: zero, one, limit, limit plus one
- Parameterized tests run one body against a table of cases
- Provide a companion object inside the test class
  - This object should provide a method that will return a list of parameter sets for the different test runs
  - Those parameter sets will be provided as constructor parameters to the class
  - The @Test function inside the class should use the parameters to execute the code and make the assertion(s)
- Test data builders hide irrelevant fields and show only what matters
- Name each case so a failure identifies the input



©Logical Imagination Group LLC

Edge Cases: Parameterized Tests and Builders

```
@RunWith(Parameterized::class)
class AmountRuleTest(private val amount: Long,
                     private val balance: Long,
                     private val ok: Boolean) {
    companion object {
        @JvmStatic @Parameterized.Parameters(name = "amount={0} balance={1} -> {2}")
        fun cases() = listOf(
            arrayOf(0L, 100L, false), arrayOf(1L, 100L, true),
            arrayOf(100L, 100L, true), arrayOf(101L, 100L, false)
        )
    }
    @Test fun validates() = assertEquals(ok, AmountRule.isValid(amount, balance))
}
```



©Logical Imagination Group LLC

Lesson 1

Skill Check ✓

1. Where does a transfer amount rule belong in the test pyramid?

A) End-to-end UI test
B) Local JVM unit test
C) Manual test only
D) Screenshot test

2. Why prefer a fake repository over a mock in ViewModel tests?

A) Fakes are faster to compile
B) Fakes test outcomes, so tests survive refactoring
C) Mocks cannot return values
D) Fakes need the network

3. Which inputs best find an off-by-one bug in an amount rule?

A) Random numbers
B) Boundary values and their neighbors
C) Only typical values
D) Only negative numbers

4. What makes a unit test non-deterministic?

A) Using assertEquals
B) Reading the real clock or network
C) Using Kotlin
D) Naming it clearly



©Logical Imagination Group LLC

Testing ViewModels, Coroutines, and Flow

- Control virtual time with runTest and test dispatchers
- Replace Dispatchers.Main in ViewModel tests with a reusable rule
- Assert Flow emissions in order with Turbine
- Verify loading, content, and error state transitions against a fake repository



©Logical Imagination Group LLC

runTest and Virtual Time

- runTest skips delays by advancing a virtual clock
- StandardTestDispatcher queues work until you advance it
- UnconfinedTestDispatcher runs eagerly, useful for simple state tests
- Never use real delays or Thread.sleep in tests



©Logical Imagination Group LLC

runTest and Virtual Time

```
@Test
fun retryBacksOffExponentially() = runTest {
    var attempts = 0
    val result = async {
        retryGet(times = 3) { attempts++; if (attempts < 3) throw IOException() else "ok" }
    }
    advanceUntilIdle()              // skips all the delays instantly
    assertEquals("ok", result.await())
    assertTrue(currentTime >= 500)  // virtual time really elapsed
}
```



©Logical Imagination Group LLC

Replacing the Main Dispatcher

- viewModelScope uses Dispatchers.Main, which does not exist on the JVM
- Set Dispatchers.setMain before the test and reset after
- Package this as a JUnit rule used by every ViewModel test
- Share one scheduler between the rule and runTest



©Logical Imagination Group LLC

Replacing the Main Dispatcher

```
class MainDispatcherRule(
    val dispatcher: TestDispatcher = StandardTestDispatcher()
) : TestWatcher() {
    override fun starting(description: Description) = Dispatchers.setMain(dispatcher)
    override fun finished(description: Description) = Dispatchers.resetMain()
}

class AccountsViewModelTest {
    @get:Rule val main = MainDispatcherRule()
    // runTest(main.dispatcher.scheduler) shares the scheduler when needed
}
```



©Logical Imagination Group LLC

Testing Flow with Turbine

- test { } collects a Flow and lets you await each item
- awaitItem suspends until the next emission or times out
- expectNoEvents proves nothing else was emitted
- StateFlow always has an initial value to consume first



©Logical Imagination Group LLC

Testing Flow with Turbine

```
@Test
fun emitsLoadingThenContent() = runTest {
    val repo = FakeAccountsRepository()
    val vm = AccountsViewModel(repo)

    vm.ui.test {
        assertEquals(emptyList<Account>(), awaitItem().accounts)   // initial
        repo.emit(listOf(anAccount("a1")))
        assertEquals("a1", awaitItem().accounts.single().id)
        cancelAndIgnoreRemainingEvents()
    }
}
```



©Logical Imagination Group LLC

Testing State Transitions End to End

- Arrange the fake, trigger the action, assert the state sequence
- Cover the happy path, the failure path, and recovery
- Assert that cached data stays visible when refresh fails
- Test one-time events separately from persistent state



©Logical Imagination Group LLC

Testing State Transitions End to End (code)

```
@Test
fun failedRefreshKeepsCachedAccountsAndShowsError() = runTest {
    val repo = FakeAccountsRepository().apply {
        emit(listOf(anAccount("a1")))
        refreshResult = ApiResult.NetworkError
    }
    val vm = AccountsViewModel(repo)
    vm.ui.test {
        skipItems(1)                    // initial snapshot
        vm.refresh()
        val refreshing = awaitItem(); assertTrue(refreshing.refreshing)
        val failed = awaitItem()
        assertFalse(failed.refreshing)
        assertEquals(1, failed.accounts.size)          // cache still visible
        assertEquals("No connection. Check your network and retry.", failed.error)
        cancelAndIgnoreRemainingEvents()
    }
}
```



©Logical Imagination Group LLC

Lesson 2

Skill Check ✓

1. Why does a ViewModel test fail on the JVM without setup?

A) Kotlin is not supported
B) Dispatchers.Main has no Android looper on the JVM
C) Hilt is missing
D) Room is missing

2. What does runTest do with delay(5000)?

A) Waits five real seconds
B) Advances a virtual clock instantly
C) Ignores it and fails
D) Runs it on the main thread

3. What is Turbine's main advantage for Flow tests?

A) It makes Flow faster
B) It asserts emissions one at a time, in order
C) It replaces Room
D) It removes the need for fakes

4. What should a test assert when a refresh fails and cached data exists?

A) The list is empty
B) The cached list is still present and an error is set
C) The app crashes
D) Nothing



©Logical Imagination Group LLC

Compose UI Tests and Espresso

- Write Compose UI tests with finders, assertions, and actions
- Design composables to be testable through state hoisting and test tags
- Handle synchronization with idling and waitUntil instead of sleeps
- Know when to use Espresso for View-based screens, intents, and interop



©Logical Imagination Group LLC

The Semantics Tree and Compose Test Rules

- Compose tests query the semantics tree, not pixels
- createComposeRule hosts a composable without an Activity
- Finders: onNodeWithText, onNodeWithTag, onNodeWithContentDescription
- Assertions and actions: assertIsDisplayed, performClick, performTextInput

Code Bite

```
class AccountRowTest {
  @get:Rule val rule = createComposeRule()
  @Test fun showsNameAndMaskedNumber() {
    rule.setContent {
       AccountRow(anAccount(), onClick = {})
    }
    rule.onNodeWithText("Wallet")
       .assertIsDisplayed()
    rule.onNodeWithText("****1234")
       .assertIsDisplayed()
     }
}
```



©Logical Imagination Group LLC

Designing Composables for Testability

- Stateless composables take state and callbacks as parameters
- A stateful wrapper connects them to the ViewModel
- Test the stateless one with fabricated state
- Use testTag only when text and descriptions are not stable



©Logical Imagination Group LLC

Designing Composables for Testability

```
@Composable
fun AccountsContent(ui: AccountsUi, onRetry: () -> Unit, onOpen: (String) -> Unit) {
    Column {
        if (ui.error != null) ErrorBanner(ui.error, onRetry)
        if (ui.refreshing) CircularProgressIndicator(Modifier.testTag("refreshing"))
        LazyColumn { items(ui.accounts, key = { it.id }) { AccountRow(it) { onOpen(it.id) }
    }
}


@Test fun bannerAppearsWithCachedRowsStillVisible() {
    rule.setContent { AccountsContent(AccountsUi(listOf(anAccount()), error = "No
connection"), {}, {}) }
    rule.onNodeWithText("No connection").assertIsDisplayed()
    rule.onNodeWithText("Wallet").assertIsDisplayed()
}
```



©Logical Imagination Group LLC

Synchronization Without Sleeping

- The Compose rule waits for recomposition and animations automatically
- waitUntil polls a condition with a timeout for async work
- Espresso uses IdlingResource to know when background work is done
- Thread.sleep in a test is a bug waiting for a slow CI machine



©Logical Imagination Group LLC

Synchronization Without Sleeping

```
@Test fun rowsAppearAfterRefresh() {
    rule.setContent { AccountsScreen(viewModel = fakeViewModel) }
    rule.waitUntil(timeoutMillis = 5_000) {
        rule.onAllNodesWithText("Wallet").fetchSemanticsNodes().isNotEmpty()
    }
    rule.onNodeWithText("Wallet").assertIsDisplayed()
}

// Espresso: register OkHttp as an idling resource so Espresso waits for requests
IdlingRegistry.getInstance().register(OkHttp3IdlingResource.create("okhttp", client))
```



©Logical Imagination Group LLC

Espresso: Views, Intents, and Interop

- Espresso drives View-based screens: onView, perform, check
- Espresso-Intents stubs and verifies external intents
- Use it for legacy screens, WebView, and Compose interop hosts
- Same discipline: no sleeps, no shared state, stable matchers



©Logical Imagination Group LLC

Espresso: Views, Intents, and Interop

```
@get:Rule val intents = IntentsRule()

@Test fun callSupportLaunchesDialer() {
    intending(hasAction(Intent.ACTION_DIAL))

       .respondWith(Instrumentation.ActivityResult(Activity.RESULT_OK, null))
    onView(withId(R.id.callSupport)).perform(click())
    intended(allOf(hasAction(Intent.ACTION_DIAL), hasData("tel:18005551234")))
}
```



©Logical Imagination Group LLC

Lesson 3

Skill Check ✓

1. What do Compose UI tests query?

A) Pixels
B) The semantics tree
C) The Gradle model
D) Logcat

2. Why hoist state out of a composable?

A) Smaller APK
B) So tests and callers can supply any state
C) Required by Hilt
D) Faster compilation

3. What replaces Thread.sleep in a UI test?

A) A longer sleep
B) waitUntil on a condition or an idling resource
C) Nothing
D) A second emulator

4. When is Espresso the right tool?

A) Never
B) For View-based screens, WebView, and verifying external intents
C) Only for unit tests
D) Only for Compose



©Logical Imagination Group LLC

Integration Tests with Hilt, Flakiness, and CI

- Replace production modules with fakes in instrumented tests using Hilt
- Write an integration test that exercises Retrofit, Room, and the repository together
- Diagnose and eliminate the common causes of flaky tests
- Run the suite on a managed device and read the reports



©Logical Imagination Group LLC

Hilt in Tests

- @HiltAndroidTest and HiltAndroidRule build a test component
- A custom test runner swaps in HiltTestApplication
- @TestInstallIn replaces a production module with a test module
- Fakes are provided at the module boundary, not inside classes



©Logical Imagination Group LLC

Hilt in Tests

```
class HiltTestRunner : AndroidJUnitRunner() {
  override fun newApplication(cl: ClassLoader?, name: String?, ctx: Context?): Application =
    super.newApplication(cl, HiltTestApplication::class.java.name, ctx)
}

@Module
@TestInstallIn(
  components = [SingletonComponent::class],
  replaces = [RepositoryModule::class]
)
object FakeRepositoryModule {
    @Provides @Singleton fun repo(): AccountsRepository =
        FakeAccountsRepository().apply { emit(listOf(anAccount())) }
}
```



©Logical Imagination Group LLC

What Integration Means

- **The boundary rule:** fake the *edges* (network, disk, time, third-party SDKs); keep everything else real.
  - That usually means fake data sources, an in-memory Room database, and real repositories, use cases, view models and composables
- **Narrow versus broad:**
  - narrow = two or three real collaborators (DAO plus database)
  - broad = a whole feature through the UI
- Locate each on the pyramid and talk about speed, diagnosis and confidence.



©Logical Imagination Group LLC

Integration: Network, Database, Repository Together

- Use the real Retrofit stack against MockWebServer
- Use a real in-memory Room database
- Assert the final observable result, not each internal call
- Cover the failure that links two layers, such as bad JSON leaving the cache intact



©Logical Imagination Group LLC

Integration: Network, Database, Repository Together

```
@Test fun malformedResponseLeavesCacheIntact() = runTest {
    dao.upsertAll(listOf(sampleEntity("a1")))
    server.enqueue(MockResponse().setBody("{bad json"))

    val result = repo.refresh()

    assertEquals(ApiResult.ParseError, result)
    assertEquals(listOf("a1"), repo.observeAccounts().first().map { it.id })
}
```



©Logical Imagination Group LLC

Taming Flaky Tests

- Causes: timing, shared state, animations, real network, test order
- Disable animations in testOptions for instrumented runs
- Use Android Test Orchestrator to isolate each test in a fresh process
- Quarantine flaky tests, then fix them; never ignore them silently



©Logical Imagination Group LLC

Taming Flaky Tests

```
// app/build.gradle.kts
android {
    defaultConfig { testInstrumentationRunner = "com.pnc.mobile.HiltTestRunner"
        testInstrumentationRunnerArguments["clearPackageData"] = "true" }
    testOptions {
        animationsDisabled = true
        execution = "ANDROIDX_TEST_ORCHESTRATOR"
    }
}
dependencies { androidTestUtil("androidx.test:orchestrator:1.5.1") }
```



©Logical Imagination Group LLC

Continuous Integration and Managed Devices

Gradle Managed Devices create and cache emulators from code

Same command locally and in CI: ./gradlew pixel6Api34DebugAndroidTest

Run unit tests on every commit; run UI tests on pull requests or nightly

Publish HTML reports and keep failing screenshots as artifacts



©Logical Imagination Group LLC

Continuous Integration and Managed Devices (code)

```
android {
    testOptions {
        managedDevices {
            localDevices {
                create("pixel6Api34") {
                    device = "Pixel 6"
                    apiLevel = 34
                    systemImageSource = "aosp"
                }
            }
        }
    }
}
// Run: ./gradlew pixel6Api34DebugAndroidTest
// Reports: app/build/reports/androidTests/managedDevice/
```



©Logical Imagination Group LLC

Lesson 4

Skill Check

1. What does @TestInstallIn do?

A) Installs the APK
B) Replaces a production Hilt module with a test module
C) Runs tests in parallel
D) Disables Hilt

2. Why is an integration test valuable in addition to mapper unit tests?

A) It is faster
B) It catches mismatches across layers such as DTO field names
C) It replaces all unit tests
D) It needs no setup

3. What is the right response to a flaky test?

A) Add a longer sleep
B) Find the unsynchronized condition or shared state and fix it
C) Delete it quietly
D) Rerun until green

4. Why define emulators with Gradle Managed Devices?

A) Smaller APK
B) Local and CI runs use the same reproducible device
C) Faster compilation
D) Required by Compose



©Logical Imagination Group LLC